

CYBERSECURITY & ETHICAL HACKING

TASK 5 – CRYPTOGRAPHY, KEY MANAGEMENT & SECURE AUTH

RabTech Academy — Cybersecurity Internship

Assessor: Pritam Bose

Cryptography & Secure Authentication Implementation Report

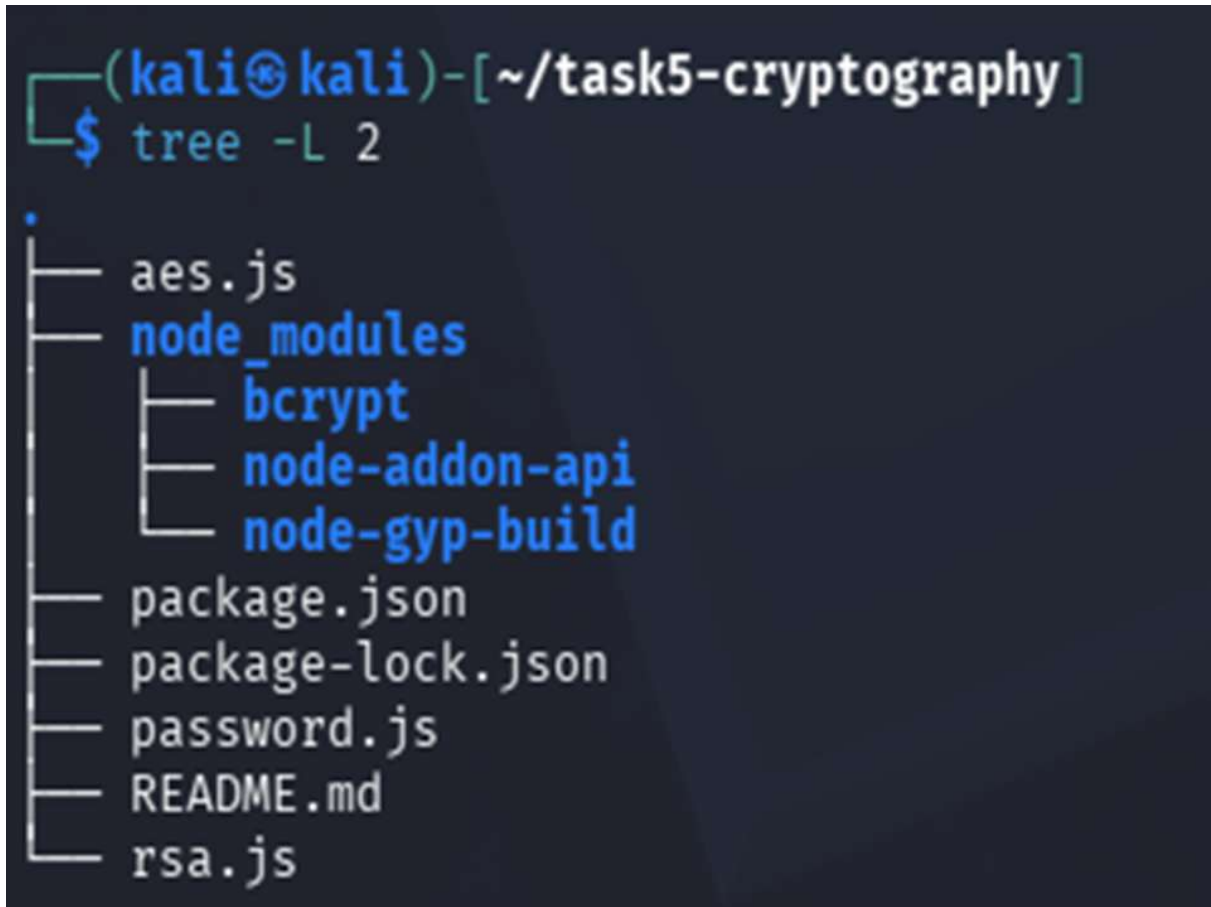
Task	Task 5 – Cryptography, Key Management & Secure Auth
Assessment Type	Cryptography & Secure Authentication Implementation
Technologies	Node.js, AES-256-GCM, RSA-2048, bcrypt
Evidence	AES encryption/decryption, RSA signature verification, password hashing
Scope	Authorized internship training environment

1. Objective

Implement and demonstrate AES-256-GCM authenticated encryption, RSA-2048 key generation and digital signatures, and bcrypt password hashing using Node.js.

2. Project Structure

The project separates AES, RSA and password-hashing implementations into individual JavaScript files.



```
(kali@kali)-[~/task5-cryptography]
$ tree -L 2
.
├── aes.js
├── node_modules
│   ├── bcrypt
│   ├── node-addon-api
│   └── node-gyp-build
├── package.json
├── package-lock.json
├── password.js
├── README.md
└── rsa.js
```

Figure 1: Task 5 project structure

3. AES-256-GCM

AES-256-GCM provides encryption together with an authentication tag for integrity verification.

```
aes.js
const crypto = require('crypto');

const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(12);
const message = 'Hello RabTech Academy';

const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
const encrypted = Buffer.concat([
  cipher.update(message, 'utf8'), cipher.final()
]);
const authTag = cipher.getAuthTag();

const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
```

```

decipher.setAuthTag(authTag);
const decrypted = Buffer.concat([
  decipher.update(encrypted), decipher.final()
]).toString('utf8');

console.log('Encrypted:', {
  iv: iv.toString('hex'),
  encrypted: encrypted.toString('hex'),
  authTag: authTag.toString('hex')
});
console.log('Decrypted:', decrypted);

```

```

(kali@kali)-[~/task5-cryptography]
$ node aes.js
Encrypted: {
  iv: 'd1c62aeda989ea15f11df128',
  encrypted: '0499fdea22102cd568ec7bcb8107d3cd9685431219',
  authTag: '0592e81167b3a00dfc0c5c6af1cee5b3'
}
Decrypted: Hello RabTech Academy

```

Figure 2: AES-256-GCM execution output

4. RSA-2048

RSA-2048 is used to generate a public/private key pair and verify a SHA-256 digital signature.

```

rsa.js
const crypto = require('crypto');

const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
  modulusLength: 2048,
  publicKeyEncoding: { type: 'spki', format: 'pem' },
  privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
});

const data = 'RabTech Academy - Task 5';

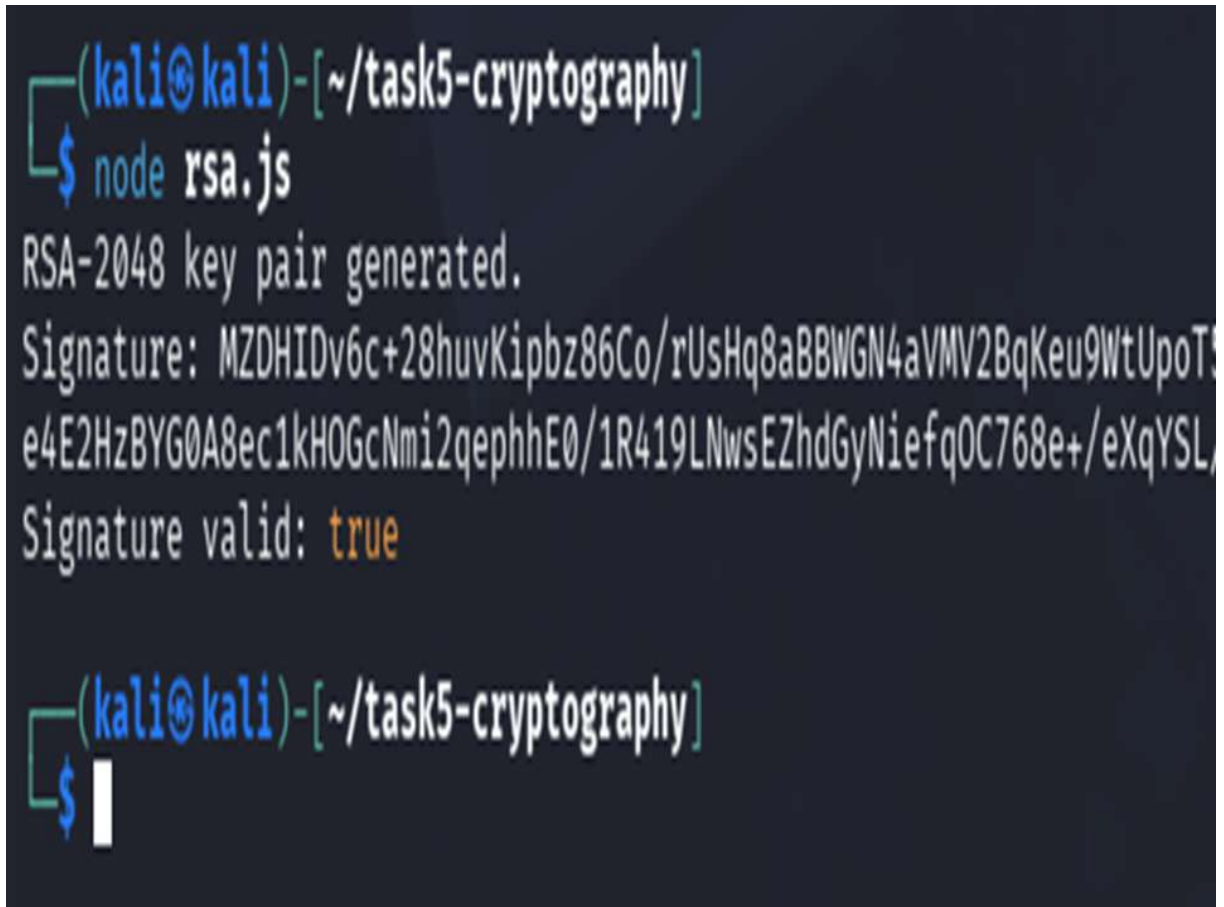
```

```
const signer = crypto.createSign('SHA256');
signer.update(data); signer.end();
const signature = signer.sign(privateKey, 'base64');

const verifier = crypto.createVerify('SHA256');
verifier.update(data); verifier.end();
const valid = verifier.verify(publicKey, signature, 'base64');

console.log('RSA-2048 key pair generated.');
```

```
console.log('Signature:', signature);
console.log('Signature valid:', valid);
```



```
(kali@kali)-[~/task5-cryptography]
$ node rsa.js
RSA-2048 key pair generated.
Signature: MZDHIDv6c+28huvKipbz86Co/rUsHq8aBBWGN4aVMV2BqKeu9WtUpOT
e4E2HzBYG0A8ec1kHOGcNmi2qephhE0/1R419LNwsEZhdGyNiefqOC768e+/eXqYSL
Signature valid: true

(kali@kali)-[~/task5-cryptography]
$
```

Figure 3: RSA-2048 execution output

5. bcrypt Password Hashing

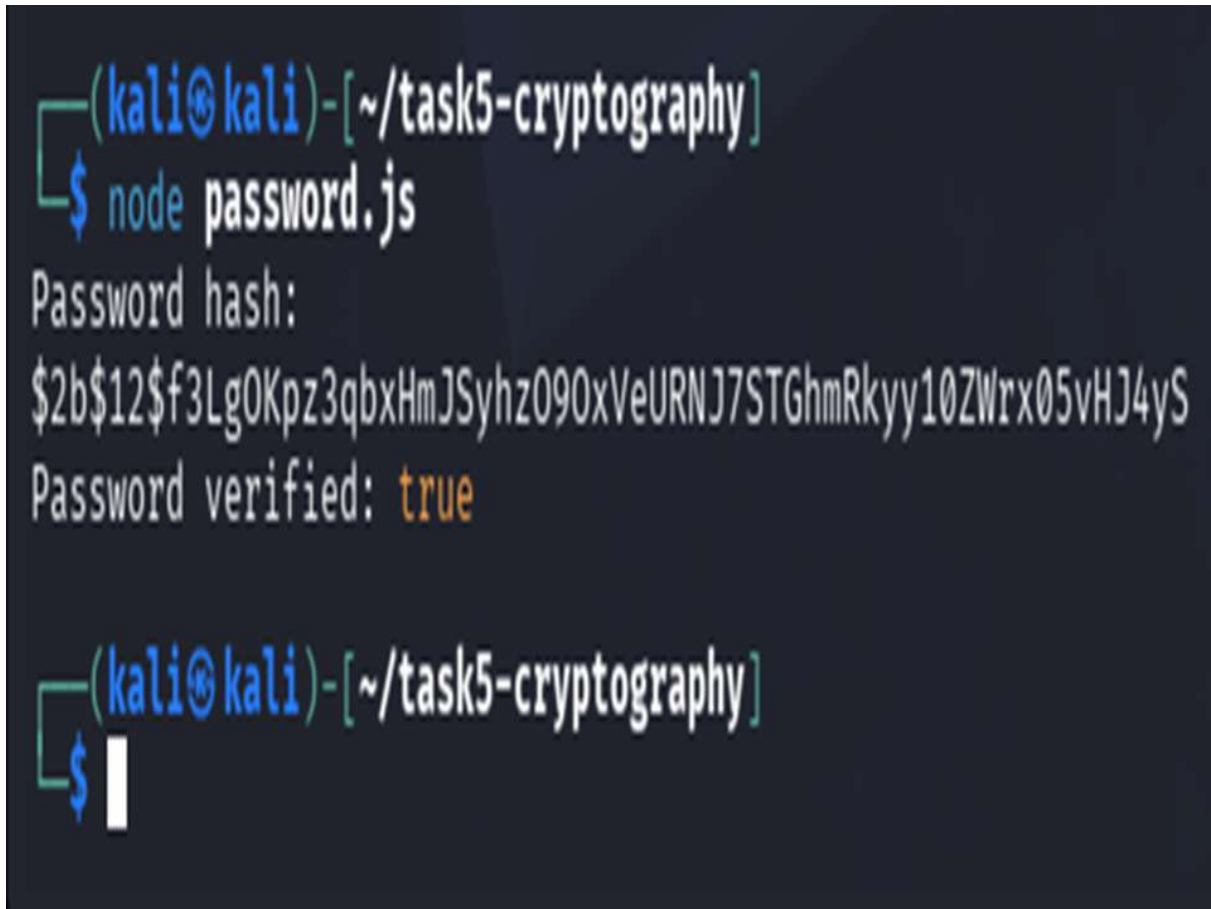
bcrypt hashes passwords with a salt and verifies the supplied password without storing plaintext.

```
password.js
const bcrypt = require('bcrypt');

const password = 'SecurePassword123!';
const saltRounds = 12;

const hash = bcrypt.hashSync(password, saltRounds);
console.log('Password hash:');
console.log(hash);
```

```
const verified = bcrypt.compareSync(password, hash);  
console.log('Password verified:', verified);
```

A terminal window with a dark background and light blue text. The prompt is (kali@kali)-[~/task5-cryptography]. The user enters \$ node password.js. The output is Password hash: \$2b\$12\$f3LgOKpz3qbxHmJSyhz090xVeURNJ7STGhmRkyy10ZWrx05vHJ4yS. Then Password verified: true. The prompt is (kali@kali)-[~/task5-cryptography]. The user enters \$.

```
(kali@kali)-[~/task5-cryptography]  
$ node password.js  
Password hash:  
$2b$12$f3LgOKpz3qbxHmJSyhz090xVeURNJ7STGhmRkyy10ZWrx05vHJ4yS  
Password verified: true  
  
(kali@kali)-[~/task5-cryptography]  
$
```

Figure 4: bcrypt hashing and verification output

6. Security / Remediation

- Do not hard-code encryption keys or private keys; use secure secret/key storage.
- Do not reuse an AES-GCM IV with the same key.
- Never store plaintext passwords; use bcrypt with an appropriate work factor.
- Keep private RSA keys protected and expose only public keys where required.
- Exclude node_modules from Git using .gitignore; keep package.json and package-lock.json.

7. Conclusion

This task successfully demonstrates the practical implementation of core cryptographic and secure authentication techniques using Node.js. AES-256-GCM was implemented for authenticated encryption and decryption, RSA-2048 was used for key generation and digital signature verification, and bcrypt was used for secure password hashing and verification. These implementations demonstrate how cryptographic controls can be applied to protect sensitive data, verify integrity, and securely handle user passwords.

Prepared by: Pritam Bose

8. References

- ❑ RabTech Academy — Cybersecurity & Ethical Hacking Internship, **Task 5: Cryptography, Key Management & Secure Auth.**
- ❑ Node.js — crypto module documentation.
- ❑ npm — bcrypt package documentation.
- ❑ OWASP — Password Storage Cheat Sheet.
- ❑ OWASP — Cryptographic Storage Cheat Sheet.

Prepared by: Pritam Bose